

Hooks

A **Hook** is a special React function that lets functional components use React features.

useState()

useState() → stores and changes data inside a component

Import the Hook in file where you use it.

```
import { useState, useEffect } from 'react';
```

Count increases when button is clicked code:

```
function App() {  
  let count = 0;  
  const increase = () => {  
    count++;  
    console.log(count);  
  };  
  return (  
    <>  
      <h1>{count}</h1>  
      <button onClick={increase}>Increase</button>  
    </>  
  );  
}
```

The following code wont work because:

But the UI **doesn't update**.

Why?

Because changing a normal JavaScript variable does **not tell React that the UI needs to be rendered again**.

ReactJS will only change the variable value in real time if they use Hooks.(useState).

Syntax

```
const [value, setValue] = useState(initialValue);
```

(value) is a variableName like: count → current value

(setValue) is a function that is used to change the value: setCount → change value

(initialValue): 0 → starting value

Example code:

```

import { useState } from "react";
function App() {

  const [count, setCount] = useState(0);

  const increase = () => {
    setCount(count + 1);
  };

  return (
    <div>
      <h1>Count: {count}</h1>

      <button onClick={increase}>
        Increase
      </button>
    </div>
  );
}

export default App;

```

```

import { useState } from "react";

function App() {

  const [name, setName] = useState("Greshi");

  return (
    <div>
      <h1>Hello {name}</h1>

      <button onClick={() => setName("Rahul")}>
        Change Name
      </button>
    </div>
  );
}

export default App;

```

useEffect()

useEffect is used when you want to perform a **side effect** in a React component.

A side effect means something that happens **outside simply calculating JSX**.

Common examples:

- API calls
- Fetching data
- Timers
- Updating document title
- Event listeners
- Subscriptions
- Running code when state changes

Syntax

```
• useEffect(() => {  
•  
•   // code to execute  
•  
• }, []);
```

➔ No dependency array

```
useEffect(() => {  
  console.log("Effect");  
});
```

Runs: After every render

➔ useEffect With Empty Dependency Array

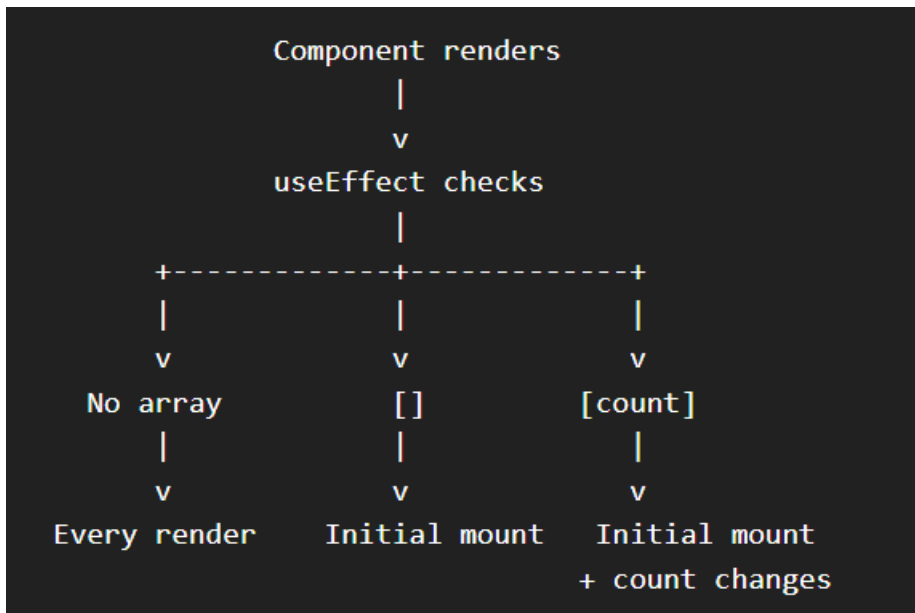
```
useEffect(() => {  
  console.log("Effect");  
}, []);
```

Runs: After initial render

➔ useEffect with Dependency Array

```
useEffect(() => {  
  console.log("Effect");  
}, [count]);
```

Runs: After initial render+When count changes



Code Example: Document title changes whenever count variable updates

```

import { useState, useEffect } from "react";

function App() {

  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `Count: ${count}`;
  }, [count]);

  return (
    <div>

      <h1>Count: {count}</h1>

      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>

    </div>
  );
}

export default App;

```

In simple words:

useState = "What data should I remember?"

useEffect = "What should I do after rendering or when this data changes?"